Monitoring Sidekiq Performance

Web application performance is much deeper than we have space for in this book. Managing it depends on the tools you are using. Nevertheless, you need a foundation on what to measure and how to think about it. In the context of Sidekiq, there are four levels where you want to be able to observe the behavior of your app, starting from highest level to lowest level:

1. *Business Function*—This an operation that our app exists to perform (in our case, creating orders). How many are created over time? How long does it take? How long did the user wait?

2. *Job Performance*—This is how specific jobs are performing. How many jobs were queued of each type? How long did they take to complete? How many failed and were retried?

3. *Sidekiq Performance*—This is about the Sidekiq subsystem as a whole. How big are the queues? How quickly are they draining?

4. *Redis*—This is the database that underpins Sidekiq. How much space is left? How many connections are being used? What is the latency between our app and Redis?

You want to *monitor* all of these layers, meaning you want to be able to consult some system to answer questions like, “How many connections were there to Redis between 5:45 and 6:13 p.m. on Monday the 4th?” or “How many orders were completed after at least one credit card decline last month?” *Alerting* on these measures is a different story, and we’ll talk about that in a bit.

In almost every case, you will need a third-party tool like Honeycomb[10] or New Relic[11] for performance monitoring. While these tools cannot automatically measure your business functions, they almost always provide out-of-the-box measurements for everything else. Despite this convenience,